

Capstone: Deploy Flask on AWS Fargate with Terraform (Simple)

This repository contains a **copy-paste ready** Terraform + Flask Docker project that meets the assignment: - VPC (2 public, 2 private) - Internet Gateway, NAT Gateway, route tables - Security Groups for HTTP/SSH - ECS Cluster (Fargate) with Task Definition and Service (2 tasks) - Application Load Balancer with target group and listener - Terraform variables and outputs

Folder structure

```
capstone-flask-ecs/
├── app/                                # Flask app + Dockerfile
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
├── terraform/
│   ├── main.tf                        # all resources (simple single-file style)
│   ├── variables.tf
│   ├── outputs.tf
│   └── terraform.tfvars.example
└── README.md                          # this file
```

1) Flask app (in `app/`)

app/app.py

```
from flask import Flask
import os

app = Flask(__name__)

@app.route('/')
def hello():
    host = os.getenv('HOSTNAME', 'unknown')
    return f"<h1>Hello from Flask on ECS Fargate!</h1><p>Host: {host}</p>"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=80)
```

app/requirements.txt

```
Flask==2.2.3
```

app/Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py ./
ENV FLASK_ENV=production
EXPOSE 80
CMD ["python", "app.py"]
```

2) Terraform (simple single-file layout)

All Terraform code is in `terraform/main.tf`, with variables and outputs split.

terraform/variables.tf

```
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
}

variable "env_name" {
  description = "Environment name (dev/prod)"
  type        = string
  default     = "dev"
}

variable "vpc_cidr" {
  description = "VPC CIDR"
  type        = string
  default     = "10.0.0.0/16"
}

variable "public_subnets" {
  description = "List of public subnet CIDRs (2)"
  type        = list(string)
  default     = ["10.0.1.0/24", "10.0.2.0/24"]
}
```

```

}

variable "private_subnets" {
  description = "List of private subnet CIDRs (2)"
  type        = list(string)
  default     = ["10.0.101.0/24", "10.0.102.0/24"]
}

variable "desired_count" {
  description = "Number of tasks"
  type        = number
  default     = 2
}

variable "container_image" {
  description = "Container image (Docker Hub or ECR). Example: mydockerhubuser/flask-ecs:latest"
  type        = string
  default     = "nginx:latest"
}

variable "container_port" {
  description = "Container port"
  type        = number
  default     = 80
}

```

terraform/main.tf

```

terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = ">= 4.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

locals {
  name_prefix = "${var.env_name}-flask-capstone"
}

```

```

# VPC
resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr
  tags = { Name = local.name_prefix }
}

# Internet Gateway
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "${local.name_prefix}-igw" }
}

# Public subnets
resource "aws_subnet" "public" {
  count                = length(var.public_subnets)
  vpc_id              = aws_vpc.main.id
  cidr_block          = var.public_subnets[count.index]
  map_public_ip_on_launch = true
  availability_zone    =
data.aws_availability_zones.available.names[count.index]
  tags = { Name = "${local.name_prefix}-public-${count.index + 1}" }
}

# Private subnets
resource "aws_subnet" "private" {
  count                = length(var.private_subnets)
  vpc_id              = aws_vpc.main.id
  cidr_block          = var.private_subnets[count.index]
  availability_zone    = data.aws_availability_zones.available.names[count.index]
  tags = { Name = "${local.name_prefix}-private-${count.index + 1}" }
}

# Availability zones data
data "aws_availability_zones" "available" {}

# Elastic IP for NAT
resource "aws_eip" "nat_eip" {
  vpc = true
}

# NAT Gateway in first public subnet
resource "aws_nat_gateway" "nat" {
  allocation_id = aws_eip.nat_eip.id
  subnet_id     = aws_subnet.public[0].id
  depends_on    = [aws_internet_gateway.igw]
  tags = { Name = "${local.name_prefix}-nat" }
}

```

```

# Route tables
resource "aws_route_table" "public_rt" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = { Name = "${local.name_prefix}-public-rt" }
}

resource "aws_route_table" "private_rt" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat.id
  }
  tags = { Name = "${local.name_prefix}-private-rt" }
}

# Route table associations
resource "aws_route_table_association" "public_assoc" {
  count          = length(aws_subnet.public)
  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public_rt.id
}

resource "aws_route_table_association" "private_assoc" {
  count          = length(aws_subnet.private)
  subnet_id      = aws_subnet.private[count.index].id
  route_table_id = aws_route_table.private_rt.id
}

# Security group for ALB (allow HTTP)
resource "aws_security_group" "alb_sg" {
  name          = "${local.name_prefix}-alb-sg"
  description    = "Allow HTTP"
  vpc_id        = aws_vpc.main.id

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
  }
}

```

```

        protocol    = "-1"
        cidr_blocks = ["0.0.0.0/0"]
    }
}

# Security group for ECS tasks (allow from ALB and allow outbound)
resource "aws_security_group" "ecs_sg" {
    name          = "${local.name_prefix}-ecs-sg"
    description   = "Allow traffic from ALB"
    vpc_id        = aws_vpc.main.id

    ingress {
        from_port     = var.container_port
        to_port       = var.container_port
        protocol       = "tcp"
        security_groups = [aws_security_group.alb_sg.id]
    }

    # SSH - optional: only for troubleshooting if you had EC2; kept here per
    requirements
    ingress {
        from_port     = 22
        to_port       = 22
        protocol       = "tcp"
        cidr_blocks   = ["0.0.0.0/0"]
    }

    egress {
        from_port     = 0
        to_port       = 0
        protocol       = "-1"
        cidr_blocks   = ["0.0.0.0/0"]
    }
}

# ALB
resource "aws_lb" "alb" {
    name          = "${local.name_prefix}-alb"
    internal      = false
    load_balancer_type = "application"
    subnets      = aws_subnet.public[*].id
    security_groups = [aws_security_group.alb_sg.id]
    tags = { Name = "${local.name_prefix}-alb" }
}

resource "aws_lb_target_group" "tg" {
    name     = "${local.name_prefix}-tg"
    port     = var.container_port

```

```

protocol = "HTTP"
vpc_id   = aws_vpc.main.id
health_check {
  path           = "/"
  interval       = 30
  healthy_threshold = 2
  unhealthy_threshold = 2
  matcher        = "200-399"
}
tags = { Name = "${local.name_prefix}-tg" }
}

resource "aws_lb_listener" "http" {
  load_balancer_arn = aws_lb.alb.arn
  port              = 80
  protocol          = "HTTP"

  default_action {
    type             = "forward"
    target_group_arn = aws_lb_target_group.tg.arn
  }
}

# IAM role for ECS Task Execution
resource "aws_iam_role" "ecs_task_exec_role" {
  name           = "${local.name_prefix}-ecs-task-exec-role"
  assume_role_policy =
data.aws_iam_policy_document.ecs_task_assume_role_doc.json
}

data "aws_iam_policy_document" "ecs_task_assume_role_doc" {
  statement {
    actions = ["sts:AssumeRole"]
    principals {
      type = "Service"
      identifiers = ["ecs-tasks.amazonaws.com"]
    }
  }
}

resource "aws_iam_role_policy_attachment" "ecs_task_exec_policy_attach" {
  role       = aws_iam_role.ecs_task_exec_role.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/AmazonECSTaskExecutionRolePolicy"
}

# ECS Cluster
resource "aws_ecs_cluster" "cluster" {

```

```

    name = "${local.name_prefix}-cluster"
}

# Task Definition
resource "aws_ecs_task_definition" "task" {
    family            = "${local.name_prefix}-task"
    cpu               = "256"
    memory            = "512"
    network_mode      = "awsvpc"
    requires_compatibilities = ["FARGATE"]
    execution_role_arn = aws_iam_role.ecs_task_exec_role.arn

    container_definitions = jsonencode([
        {
            name        = "flask-app"
            image        = var.container_image
            cpu          = 128
            memory       = 256
            essential    = true
            portMappings = [
                {
                    containerPort = var.container_port
                    protocol       = "tcp"
                }
            ]
            logConfiguration = {
                logDriver = "awslogs"
                options = {
                    awslogs-group      = "/ecs/${local.name_prefix}"
                    awslogs-region     = var.aws_region
                    awslogs-stream-prefix = "ecs"
                }
            }
        }
    ])
}

# CloudWatch Log Group
resource "aws_cloudwatch_log_group" "ecs_log_group" {
    name           = "/ecs/${local.name_prefix}"
    retention_in_days = 7
}

# ECS Service
resource "aws_ecs_service" "service" {
    name            = "${local.name_prefix}-service"
    cluster         = aws_ecs_cluster.cluster.id
    task_definition = aws_ecs_task_definition.task.arn

```



```

desired_count    = var.desired_count
launch_type      = "FARGATE"

network_configuration {
  subnets          = aws_subnet.private[*].id
  security_groups   = [aws_security_group.ecs_sg.id]
  assign_public_ip  = false
}

load_balancer {
  target_group_arn = aws_lb_target_group.tg.arn
  container_name   = "flask-app"
  container_port   = var.container_port
}

depends_on = [aws_lb_listener.http]
}

# Allow TG to register targets from ENIs - AWS manages this automatically with
# Fargate

# Outputs are in outputs.tf

```

terraform/outputs.tf

```

output "alb_dns_name" {
  description = "ALB DNS name (public endpoint)"
  value       = aws_lb.alb.dns_name
}

output "ecs_cluster_name" {
  description = "ECS cluster name"
  value       = aws_ecs_cluster.cluster.name
}

output "running_task_count" {
  description = "Number of running tasks (desired)"
  value       = aws_ecs_service.service.desired_count
}

```

terraform/terraform.tfvars.example

```

aws_region    = "us-east-1"
env_name      = "dev"

```

```
container_image = "<your-dockerhub-or-ecr-image>" # e.g. myuser/flask-ecs:latest
```

3) How to build & push the Flask image (use Docker Hub or ECR)

Option A — Docker Hub (fastest if you have Docker Hub account)

1. Build:

```
cd app
docker build -t yourdockerhubuser/flask-ecs:latest .
```

1. Login & push:

```
docker login
docker push yourdockerhubuser/flask-ecs:latest
```

1. Update `terraform/terraform.tfvars` with `container_image = "yourdockerhubuser/flask-ecs:latest"`

Option B — Amazon ECR (works well but needs AWS creds)

```
# create repo (once)
aws ecr create-repository --repository-name flask-ecs --region us-east-1

# build & tag
docker build -t flask-ecs:latest .
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin <aws_account_id>.dkr.ecr.us-east-1.amazonaws.com
docker tag flask-ecs:latest <aws_account_id>.dkr.ecr.us-east-1.amazonaws.com/flask-ecs:latest
docker push <aws_account_id>.dkr.ecr.us-east-1.amazonaws.com/flask-ecs:latest
```

4) Terraform apply (once you have AWS credentials)

1. Configure AWS CLI with an IAM user that has sufficient privileges (VPC, EC2, IAM, ECS, ALB, ECR, CloudWatch).

2. Initialize & apply:

```
cd terraform
terraform init
terraform plan -var-file="terraform.tfvars"
terraform apply -var-file="terraform.tfvars" -auto-approve
```

1. After apply, Terraform will print outputs including `alb_dns_name`. Test with:

```
curl http://<alb_dns_name>
```

5) If you DON'T have AWS credentials (you said No)

Two quick options so you can still submit and demo:

- **Option 1 (recommended for submission now):** Push the Docker image to Docker Hub and attach the Terraform code + README as your project submission. In your README, clearly state that the code is ready to run and list the exact commands to run once credentials are available. Provide sample Terraform outputs (you can provide mocked example outputs) and screenshots of a local Docker run. This shows you implemented IaC correctly.
- **Option 2 (local demo):** Run the Flask container locally with Docker Compose and show it's working, and still submit Terraform code. Example `docker-compose.yml`:

```
version: '3.8'
services:
  web:
    build: ./app
    ports:
      - 8080:80
```

Run:

```
docker compose up --build
# then open http://localhost:8080
```

Include a short video or a few screenshots of the local running app and the `terraform` folder in your submission.

6) Security notes & minimal permissions

- The IAM user that runs Terraform needs policies for: VPC, EC2, ECS, IAM (for roles), ELB, CloudWatch Logs, ECR (if used).
 - For safety, avoid using overly broad policies in production; grant least privilege.
-

7) What I already included

- Ready-to-run Flask app and Dockerfile
 - Full Terraform template (VPC, subnets, NAT/IGW, security groups, ALB, ECS Fargate service, task definition, IAM execution role, CloudWatch log group)
 - Variables + outputs
 - Example commands to build/push image and run Terraform
-

If you want, I can now: - (A) Generate a `terraform.tfvars` file with example values for you to edit, - (B) Produce a short `submission.md` / `report.txt` you can upload (explaining architecture and commands), or - (C) Walk you through how to create an AWS IAM user and minimum policy JSON to run this (I can paste the policy).

Tell me which of A / B / C you want next and I will add it immediately.